HW

Function Pointer, Dynamic Memory Allocation, Multi Dimensional Array, Pointer and String

[Problem1]

Goal:

- **Implement class Person by fill in the blank part of the code**
- **Implement class which saves student's Name, Student ID, and Age**

Member function

1. init()

Implement the init() member function.

What this function should do

This function must initialize the member variables of the class:

- copy the input name into the class variable for name
- save the input student ID into the class variable for student ID
- save the input age into the class variable for age

2. printStudentInfo()

Implement the printStudentInfo() member function.

What this function should do

This function must print the student information currently stored in the object:

- Name
- Student ID
- Age

Make sure the output format matches the expected result.

3. getID()

Implement the getID() member function.

What this function should do

Return the student ID stored in the object.

4. getAge()

Implement the getAge() member function.

What this function should do

Return the age stored in the object.

5. getName()

Implement the getName() member function.

What this function should do

Return the name stored in the object.

Implement the following function:

6. void personSwap(...)

This function receives Person objects as input and swaps their stored values.

After the function is called, the two input objects must exchange all of their information:

- Name
- Student ID
- Age

In the attached file, correct the marked //*problem***// to complete it.**

[Result]

```
Before Swap:
Name: Alice, ID: 2026312345, Age: 20
Name: Bob, ID: 2019354321, Age: 25

After Swap:
Name: Bob, ID: 2019354321, Age: 25
Name: Alice, ID: 2026312345, Age: 20
```

[Problem2]

Goal:

- **mplement a small seat arrangement management system using a dynamically allocated 10x10 two-dimensional integer array.**

You must use dynamic memory allocation for the 2D array, initialize it with memset, copy it with memcpy, and release all allocated memory at the end. This problem is based on the lecture contents on dynamic memory allocation and multi-dimensional arrays.

Rules

- **The seat map size is fixed to 10 x 10.**
- **Do not receive any runtime input.**
- **Empty seat value is 0.**
- **Assigned seat value is the last two digits of a student ID.**
- **Print every number in two digits (00, 11, 24, ...).**
- **Use dynamic allocation for the 2D array.**
- **Use a function print_seat that receives only the array and its size.**
- **Do not use global variables.**

1. Dynamically allocate a 10x10 integer array.
Initialize the whole array to 0 using memset.
Then assign the following values to the seat map

- (0, 0) = 11
- (0, 3) = 24
- (1, 1) = 35
- (2, 5) = 47
- (3, 3) = 58
- (4, 9) = 69
- (5, 0) = 70
- (5, 5) = 81
- (5, 9) = 92
- (6, 4) = 13
- (7, 7) = 26
- (8, 2) = 39
- (9, 9) = 44

2. Implement the following function

void print_seat(int **seat, int row, int col);

This function must print the full seat map in matrix form.

Each value must be printed in two digits, separated by one space.

Then Print using print_seat() :

```
[Original Seat Map]
11 00 00 24 00 00 00 00 00 00
00 35 00 00 00 00 00 00 00 00
00 00 00 00 00 47 00 00 00 00
00 00 00 58 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 69
70 00 00 00 00 81 00 00 00 92
00 00 00 00 13 00 00 00 00 00
00 00 00 00 00 00 00 26 00 00
00 00 39 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 44
```

3. Create another dynamically allocated 10x10 integer array named backup.

Copy the original seat map into backup using memcpy.

You must copy row by row.

Then Print :

```
[Backup Seat Map]
11 00 00 24 00 00 00 00 00 00
00 35 00 00 00 00 00 00 00 00
00 00 00 00 00 47 00 00 00 00
00 00 00 58 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 69
70 00 00 00 00 81 00 00 00 92
00 00 00 00 13 00 00 00 00 00
00 00 00 00 00 00 00 26 00 00
00 00 39 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 44
```

4. In the original seat map only, clear the entire row 5 by setting all values in that row to 0.

Then, print :

```
[Original Seat Map After Clearing Row 5]
11 00 00 24 00 00 00 00 00 00
00 35 00 00 00 00 00 00 00 00
00 00 00 00 00 47 00 00 00 00
00 00 00 58 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 69
00 00 00 00 00 00 00 00 00 00
00 00 00 00 13 00 00 00 00 00
00 00 00 00 00 00 00 26 00 00
00 00 39 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 44
```

and print backup :

```
[Backup Seat Map After Original Was Modified]
11 00 00 24 00 00 00 00 00 00
00 35 00 00 00 00 00 00 00 00
00 00 00 00 00 47 00 00 00 00
00 00 00 58 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 69
70 00 00 00 00 81 00 00 00 92
00 00 00 00 13 00 00 00 00 00
00 00 00 00 00 00 00 26 00 00
00 00 39 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 44
```

This step must clearly show that the copied array is independent from the original array.

5. Using the backup seat map, find the row that has the largest number of assigned seats (non-zero values).

If there is a tie, print the smallest row index.

Then, print :

```
[Row With Most Assigned Seats]
Row: 5
Count: 3
```

Finally, release all dynamically allocated memory for both arrays.

[Problem3]

Goal:

- **Implement a simple string dictionary using dynamically allocated memory.**

You must store words using char*, allocate memory separately for each word, copy strings properly, search, modify, delete, and free all allocated memory. This problem is based on the lecture contents on pointer/string handling and dynamic allocation for strings.

Rules

- **Do not receive any runtime input.**
- **Do not use std::string, vector, or other STL containers for storing the words.**
- **Use a char **list structure.**
- **Allocate memory for each word with the exact required size (strlen(word) + 1).**
- **Use string copy functions such as strcpy / strcpy_s.**
- **After deletion, shift all following elements one position to the left.**

1. Create a string list that can store up to 5 words.

Store the following words in this exact order:

- **alpha**
- **bravo**
- **charlie**
- **delta**
- **echo**

Each word must be stored in its own dynamically allocated memory space.

Then print:

```
[Initial Dictionary]
[0] alpha
[1] bravo
[2] charlie
[3] delta
[4] echo
```

2. Search for a word

- **delta**

If the word exists, Print :

```
[Search Result]
delta found at index 3
```

3. Modify the word at index1 from ‘bravo’ to ‘blueberry’

Then print :

```
[After Update]
[0] alpha
[1] blueberry
[2] charlie
[3] delta
[4] echo
```

4. Delete the word ‘charlie’

Then print :

```
[After Delete]
[0] alpha
[1] blueberry
[2] delta
[3] echo
```